ORACLE CLOUD INFRASTRUCTURE

Behavioral Interview Prep Pack

STAR answers and follow-ups for all ten OCI Core Values

Dev Kumar | Principal Software Engineer, Core Infrastructure | Nashville, TN (Req. 343056)

How to use this pack

Ten distinct stories, one per OCI Core Value, with no overlap between them. Each is drawn from a different part of your resume so that across a full interview loop you never repeat yourself, even when four different interviewers probe four different values.

Each answer is written to be spoken in roughly 90 to 120 seconds. Read the Situation and Task almost verbatim; they set up the story. Do not recite the Action bullets in order. Lead with the two or three that matter most and let the interviewer pull the rest out through follow-ups, which is what the prep pack means when it says an interview is a conversation.

The follow-up questions are the ones a Principal-level OCI interviewer is most likely to ask, because they test whether you actually did the work. Answer them with the specific numbers and mechanisms given here. Oracle's own guidance is explicit that they want detail and will keep drilling; generalizations are what sink candidates at this level.

Two habits to hold to throughout: always land the Result with a number, and always close with what you learned or would do differently. Several OCI values (Own without ego, Take risks remain calm, Act now iterate) are effectively scored on whether you can describe a failure without becoming defensive about it.

Story index

Use this to keep your stories separated in your head under pressure. If an interviewer asks a question that could fit two values, pick the story you have not yet used in that loop.

#	Core Value	Your Story	Headline Result
1	**Put Customers First**	Fixing hallucinated recommendations at Sprouts AI instead of shipping the workaround customers asked for	35% production accuracy lift; adoption to 1K+ daily users
2	**Act Now, Iterate**	Building a deliberately ugly CI/CD v1 at Resilience Inc rather than waiting for a funded platform initiative	Release frequency doubled; zero-downtime deploys; deploy window 2 hours to under 10 minutes
3	**Nail the Basics**	Refusing to build a 'smart' auto-repair service at Resilience Inc when the real gap was idempotency	State-corruption incidents to zero; a quarter of planned work avoided

#	Core Value	Your Story	Headline Result
4	**Expect and Embrace Change**	Pivoting Sprouts AI's bespoke tool adapters to MCP mid-quarter while a large customer commitment was live	Integration cost from ~3 days to under 1 day; 10 integrations delivered in 5 weeks
5	**Innovate Together**	Building the shared Agentstore orchestration platform from three teams' competing designs, on a junior engineer's topology	Three teams on one substrate; duplicate agent infrastructure eliminated
6	**Take Risks, Remain Calm**	Running brownout and fault-injection exercises, including a bounded production run, before signing a 99.99% commitment	Four latent failure modes found pre-customer; 99.99% sustained
7	**Own Without Ego**	Being wrong about the OKE deployment topology at Sprouts AI, saying so in writing, and rebuilding on the reviewer's design	p99 held sub-second under concurrent batch load; least-privilege model became the team template
8	**Earn Trust, Give Trust**	Breaking the two-person incident hero pattern at Resilience Inc by handing real authority to newer engineers	MTTR down 40%; on-call from 2 responders to 7
9	**Take Pride in Your Work**	Rebuilding a nightly manual reconciliation at Tata Steel that nobody had asked me to touch	60% reduction in manual processing time; ~40% faster APIs at 99.99% uptime
10	**Challenge Ideas, Champion Execution**	Challenging 'internal-only is safe' on a multi-tenant platform at Tata Steel, then owning the remediation to closure	All identified gaps closed before onboarding; passed internal audit

CORE VALUE 1 OF 10

Put Customers First

Story: *Fixing hallucinated recommendations at Sprouts AI instead of shipping the workaround customers asked for*

Situation. At Sprouts AI our agentic go-to-market platform was serving roughly a thousand daily users. Over about three weeks, several enterprise customers raised the same complaint: the account and contact recommendations were plausible but wrong. The agent was confidently returning entities that did not exist in their CRM. What the customers explicitly asked for was a thumbs-down button and a manual override so their sales reps could correct bad results themselves.

Task. I owned recommendation quality for the platform. My goal was to restore trust in the output. The easy path was to build exactly what they asked for in a sprint and close the tickets. I did not think a correction button was the right thing for them, because it moved the cost of our defect onto their reps, several hours a week each.

Action.

- Sat in on eight customer calls over two weeks and asked reps to walk me through recommendations they had rejected, so I was looking at real failures rather than synthetic ones.
- Sampled roughly 300 failed recommendations and hand-tagged the failure modes. About 60% were retrieval misses, where the right record existed but was never pulled into context; the rest split between prompt and output-format problems.
- Rebuilt retrieval as RAG on PostgreSQL with pgvector, using hybrid retrieval that combined vector similarity with keyword matching, and a chunking strategy tuned per document type rather than one global chunk size.
- Moved the model to structured outputs against a strict schema, so downstream services stopped parsing free text and malformed responses failed loudly instead of silently.
- Added a grounding guardrail: if no retrieved chunk cleared the similarity threshold, the agent returns 'I don't have data for that' rather than generating. Saying nothing beats saying something wrong to a customer.
- Built an offline evaluation harness with a golden set of about 500 labeled queries that ran in CI, so an accuracy regression blocked the merge rather than reaching production.
- Shipped the thumbs-down control they had asked for, but wired every downvote into the evaluation set instead of a ticket queue, so their corrections improved the system rather than just logging a complaint.

Result. Production recommendation accuracy improved 35%. Hallucination-class complaints went to effectively zero within a quarter, adoption grew past a thousand daily users at sub-second latency, and two of the accounts that had escalated expanded their seat count at renewal. What I took away is that customers describe a symptom and ask for a workaround, and that both are useful information. Their ask told me where the pain was. It did not tell me what to build.

Likely follow-up questions

Q1. Why not just build the override they asked for? They are the customer.

A. I did build it, but I refused to let it be the whole answer. Putting customers first means doing the right thing for them ahead of doing exactly what they specify. An override would have made their reps permanently responsible for correcting our defect, which is a tax we would have charged them forever. I also went back to them with what I found in the failure sample and told them the honest version: the root cause is retrieval, here is the plan, here is the interim control. Nobody objected once they saw the data.

Q2. How did you actually measure accuracy? 'Accuracy' for a generative system is slippery.

A. Two layers. For retrieval I measured precision and recall at k against the golden set, which is deterministic and cheap. For end-to-end answer quality I used a labeled rubric with LLM-as-judge, and I calibrated the judge against human labels on a held-out slice, checking agreement before I trusted any number it produced. The 35% figure is the end-to-end rubric score. I reported the retrieval numbers alongside it because they are the ones I would defend hardest.

Q3. Hybrid retrieval plus reranking adds latency. How did you keep it sub-second?

A. I set an explicit p99 request budget and split it per stage. Embeddings for the query were cached, pgvector ran an HNSW index so vector search stayed in single-digit milliseconds at our corpus size, and I retrieved a wide candidate set of about 50, then reranked down to eight before generation. The reranker was the expensive stage, so it was bounded by both candidate count and a hard timeout that fell back to raw similarity order. Generation dominated the budget, which is where I spent the remaining headroom.

Q4. What would you do differently knowing what you know now?

A. Build the evaluation harness first. I spent the first two weeks improving retrieval on intuition and could not prove which change caused what. Once the golden set existed, iteration got dramatically faster. I would also instrument retrieval quality separately from generation quality from day one, because when they are collapsed into one metric you cannot tell whether the model is reasoning badly or was simply never handed the right data.

CORE VALUE 2 OF 10

Act Now, Iterate

Story: *Building a deliberately ugly CI/CD v1 at Resilience Inc rather than waiting for a funded platform initiative*

Situation. At Resilience Inc, releases were manual. An engineer would SSH into hosts, run database migrations by hand, and restart services inside a roughly two-hour window every other week. Configuration had drifted between staging and production to the point that about a third of our incidents traced back to an environment difference. Everybody agreed it was bad. A 'platform modernization' proposal had been written and had sat unfunded for a full quarter.

Task. This was not my assigned project and nobody had asked me to fix it. I was on the on-call rotation, so I was absorbing the pain directly, and I decided that waiting for the initiative to get staffed was itself a decision I did not want to make.

Action.

- Timeboxed a v1 to a few evenings: a single GitHub Actions workflow that built, tested, and deployed one low-risk internal service. It was hardcoded, handled exactly one service, and had no secrets management. It worked.
- Demoed it in a team meeting rather than writing a proposal about it, and framed it explicitly as a starting point, listing what I had deliberately left out.
- Iterated in four increments after that. First, containerized the service and moved to Kubernetes rolling deployments with readiness and liveness probes so traffic only reached healthy pods.
- Second, moved infrastructure into Terraform with reusable modules parameterized per environment, so dev, staging, and production were the same code with different variables and drift stopped being possible.
- Third, added migration gating and rollback tooling. Migrations followed expand-and-contract so every release was backward compatible with the previous image, which is what makes a rollback actually safe rather than theoretically safe.
- Fourth, rolled the pattern out to the remaining services with a one-page runbook, and paired with two engineers so they onboarded their own services rather than filing requests with me.

Result. Release frequency doubled, deploys became zero-downtime, and the deploy window went from around two hours to under ten minutes. Environment-drift incidents essentially disappeared from our postmortems. The unfunded modernization proposal was eventually closed as already delivered. The lesson I carry is that a grungy working version starts a real conversation, and a perfect design document starts a debate.

Likely follow-up questions

Q1. What did you deliberately leave out of v1, and how did you keep that from becoming permanent debt?

A. I left out secrets rotation, canary deploys, multi-region, and any service other than the one. I wrote those four items into the demo slide as known gaps with a rough ordering, and I tracked them as real backlog tickets rather than a paragraph in a doc. Three of the four got done within two months. Multi-region never did,

because by then it genuinely was not the highest-value work, and I would rather retire an item honestly than pretend it is still planned.

Q2. How did you get buy-in for this without any authority to mandate it?

A. I brought two things: the working demo, and a count from our own incident history of how many incidents in the prior six months had an environment difference in the timeline. That number was hard to argue with because it came from postmortems the team had written itself. Then I made adoption opt-in and did the migration work alongside whoever owned the service, so for them it was less effort than staying put. Nobody had to be convinced in the abstract.

Q3. Moving that fast on the deploy path is risky. What was your safety margin?

A. I started with the lowest-blast-radius service we had, an internal tool with no customer traffic. Every increment kept the old manual path working in parallel until the new one had run clean for a couple of weeks. Rollback was one command back to the previous image, which was only trustworthy because of the migration discipline. I was moving quickly but the ordering was chosen so the first mistake would be cheap.

Q4. You mentioned expand-and-contract. Walk me through it.

A. A schema change ships in phases. First release only adds: new column nullable, new table, dual writes if needed, with the application reading from the old shape. Once that is deployed and stable, a second release moves reads over. Only a third release drops the old column. At no point is the current image incompatible with the previous one, which is precisely the property a rollback depends on. The cost is three releases instead of one, and I think that is cheap for the guarantee it buys.

CORE VALUE 3 OF 10

Nail the Basics

Story: *Refusing to build a 'smart' auto-repair service at Resilience Inc when the real gap was idempotency*

Situation. Our Kafka-based event pipeline at Resilience Inc was periodically corrupting records: duplicate charges, counters that drifted, status fields that regressed. The team's proposed fix was an auto-reconciliation service that would detect corrupted records and repair them on a schedule. It had been scoped at roughly a quarter of work for two engineers and had visible support because it sounded sophisticated.

Task. I had a strong suspicion we were about to build a repair layer on top of a guarantee we had never actually implemented. Before agreeing to the plan, I wanted to establish what was really causing the corruption.

Action.

- Pulled distributed traces and consumer-lag metrics for three known corruption incidents and lined them up against broker events. All three overlapped with consumer group rebalances or brief network partitions.
- Established the actual mechanism: our consumers were at-least-once, which is correct and expected, but our handlers were not idempotent. A redelivery after a timeout re-applied the write. Nothing exotic, just a missing fundamental.
- Fixed the basics rather than compensating for them. Derived a natural idempotency key per event from producer-supplied identifiers, and enforced it with a unique constraint in Postgres as the durable source of truth, with a Redis dedup set with TTL in front as the fast path.
- Made handlers commutative where the domain allowed it, and keyed partitioning so events for the same entity always landed on the same partition and were processed in order.
- Added rate limiting and backpressure so a slow downstream paused consumption rather than letting an unbounded in-memory queue build up and then die with the pod.
- Added a dead-letter queue with replay tooling, and wrote a test suite that replayed duplicated, reordered, and delayed event streams against every handler as part of CI.
- Took the findings to the team with the trace evidence and proposed we defer the reconciliation service by one month and see whether it was still needed.

Result. State-corruption incidents went to zero and stayed there, and we consistently met our SLOs through subsequent rebalances. The reconciliation service was never built, which saved roughly a quarter of engineering time for two people. The habit I took from it is to ask, when an advanced solution is proposed, whether it is compensating for a basic guarantee we skipped.

Likely follow-up questions

Q1. You used Redis for deduplication. What happens when Redis is unavailable or evicts a key?

A. Nothing correctness-relevant, by design. Redis is a fast path, not the source of truth. On a Redis miss or failure the handler falls through to the database insert, where the unique constraint rejects the duplicate and we treat the constraint violation as a successful no-op. Redis only exists to keep us from hammering the

database with duplicates during a redelivery storm. I would not put a correctness guarantee behind a cache with an eviction policy.

Q2. Why not pursue exactly-once semantics instead?

A. Kafka transactions give you exactly-once within Kafka, but our handlers wrote to Postgres and called external services, so the atomicity would have stopped at the boundary that mattered. Achieving it end-to-end would have meant a transactional outbox and a good deal of coordination complexity. At-least-once delivery plus idempotent handlers gets the same observable behavior for far less machinery, and it degrades more predictably. If we had needed multi-record atomic units, I would have revisited that.

Q3. Describe how backpressure actually worked.

A. Bounded work queues rather than unbounded ones, and when the queue hit its high-water mark the consumer paused its partition assignments instead of continuing to poll. That let lag build visibly in Kafka, where it was measurable and alertable, rather than invisibly in process memory where it eventually turned into an OOM kill. We alerted on consumer lag and its rate of change, and for non-critical event classes we shed load rather than queued it.

Q4. How did you convince a team that had already committed to building the reconciliation service?

A. I did not argue about the design. I brought the three traces and asked the team to explain the corruption mechanism to me without invoking idempotency, and they could not, because the mechanism was plainly redelivery. Then I proposed something small: defer for one month while I did the idempotency work, and revisit. That was easy to say yes to because it was reversible. After a month with zero incidents, the decision made itself.

CORE VALUE 4 OF 10

Expect and Embrace Change

Story: *Pivoting Sprouts AI's bespoke tool adapters to MCP mid-quarter while a large customer commitment was live*

Situation. At Sprouts AI we had built LLM tool calling in-house. Each of our roughly twenty tools was a bespoke adapter with its own authentication, schema definition, retry logic, and error handling, and adding a new one cost about three days. Mid-quarter two things landed at once: MCP gained real traction and two of our partner vendors shipped MCP servers, and our largest customer committed us to ten new integrations in six weeks. My roadmap for the quarter was no longer the roadmap.

Task. I had to decide quickly whether to keep grinding out adapters or adopt an emerging standard, without stalling a commitment that was already made and without betting the platform on something that might not last.

Action.

- Timeboxed a three-day spike rather than opening a design debate. The constraint was that the spike had to produce a runnable comparison, not a recommendation.
- Built an MCP client layer behind our existing internal tool interface, using an adapter so nothing above the interface changed. That decision mattered more than the MCP decision itself, because it meant the change was reversible.
- Migrated three representative tools and measured concrete numbers: lines of integration code, time to add a tool, and how many error cases we had to handle ourselves versus got from the protocol.
- Proposed a two-track plan rather than a rewrite: every net-new integration goes through MCP, existing adapters migrate opportunistically when they are touched for other reasons. No big-bang migration, no freeze.
- Took the opportunity to rebuild orchestration on LangGraph at the same time, so tool selection, retry policy, and human-in-the-loop escalation became explicit graph nodes with their own state, instead of behavior buried in prompt text.
- Wrote down what would make us reverse course: if MCP adoption stalled among vendors, or the spec churned in a breaking way twice, we would freeze the client layer and go back to adapters behind the same interface.

Result. Integration cost dropped from about three days to under one, and we delivered the customer's ten integrations in five weeks against a six-week commitment. Because the client sat behind our own interface, later provider swaps became configuration changes rather than projects. The LangGraph rebuild gave us per-node tracing, which paid off immediately on-call. My takeaway is that the way to absorb a change you did not plan for is to isolate it behind an interface first and then decide with data, rather than deciding on conviction.

Likely follow-up questions

Q1. MCP was new. How did you justify betting on something that could churn?

A. I did not bet on it. I bounded the exposure. The MCP client sat behind an interface we controlled, which meant the blast radius of being wrong was one layer and a few weeks of work, not the platform. I also wrote the reversal criteria down before adopting, which forces you to be honest about what evidence would change your mind. Taking a risk on an emerging standard is reasonable; taking one you cannot exit is not.

Q2. What broke or got worse in the transition?

A. Per-tool rate limiting. We had hand-tuned limits inside each adapter, and moving to a uniform client meant re-implementing that at the client layer with per-server configuration. We shipped without it for about a week and tripped a partner's rate limit in staging, which is where I would rather find it. We also lost some very specific error messages that our adapters had produced, so debugging got slightly worse until we added structured error mapping.

Q3. How did you communicate a mid-quarter pivot to stakeholders who had already been told a different plan?

A. Directly and early, with the delivery date as the headline rather than the technology. The message was: the commitment is ten integrations in six weeks, the adapter path arithmetically does not reach that, here is the spike result, here is the fallback if it fails. Leading with the customer outcome kept it from being a debate about frameworks. I also gave a checkpoint date at week two where I would report actual numbers, so nobody had to take it on faith.

Q4. How did moving to LangGraph change your operational picture?

A. Substantially, and mostly through observability. When orchestration lives in prompt text, a bad run is one opaque blob. With explicit nodes, each step emits its own span with inputs, outputs, latency, and retry count, so on-call can see which node failed rather than that the agent failed. It also made state serializable, so we could replay a failed run deterministically against a fix instead of trying to reproduce it by prompting.

CORE VALUE 5 OF 10

Innovate Together

Story: *Building the shared Agentstore orchestration platform from three teams' competing designs, on a junior engineer's topology*

Situation. Three teams at Sprouts AI, mine included, were each independently building agent loops. Every one of us had written our own memory layer, tool registry, and ad-hoc evaluation. Nobody owned the shared problem, and every team's stated reason for not consolidating was that their use case was special. I had a design in mind for a shared substrate and could have written it and pushed it.

Task. I wanted a common platform, but a platform that one person imposes gets adopted by nobody and quietly forked. My real task was to get a design the other teams would defend as theirs.

Action.

- Ran a working session with engineers from all three teams, a product manager, and a support engineer. I invited the support engineer deliberately, because she handled the escalations and knew the failure modes none of the builders saw.
- Brought three rough sketches instead of one proposal, and explicitly asked people to break them. Framing it as options rather than a recommendation is what made it safe to disagree with me.
- A junior engineer proposed a supervisor-and-worker topology I had initially dismissed as overkill. In the room it became clear it directly answered the 'my use case is special' objection, because each team would own its own worker agent behind a common supervisor contract. I said so, and we built his design rather than mine.
- Built Agentstore on LangGraph as the shared substrate: supervisor-worker orchestration, tool calling, shared memory, MCP server integration, and evaluation hooks each team could plug their own cases into.
- Turned the support engineer's escalation history into a failure taxonomy, which became the seed evaluation suite. That gave every team a shared definition of 'worse than before'.
- Made adoption opt-in with contribution docs and clear extension points, and paired with each team on their first worker rather than handing them a README.
- Open-sourced the MCP integration piece after a security and legal review, which raised the quality bar on the interfaces because they were going to be read by strangers.

Result. All three teams moved onto the one substrate, the duplicated memory and tool-registry code was deleted, and evaluation hooks meant a regression in one team's worker was caught before release rather than in an escalation. The strongest architectural decision in the platform was not mine, and the platform is better for it. That is the part I would repeat.

Likely follow-up questions

Q1. What if a team had refused to adopt it?

A. That was a real possibility and it is why adoption was opt-in with explicit escape hatches, including a way to run a worker with a custom tool registry. My view is that a platform earns adoption or it should not have it. If a

team had stayed out, that would have been signal about a gap in the substrate, and I would have gone and found out what it was rather than escalating to get them mandated onto it.

Q2. Describe the supervisor-worker topology technically.

A. A supervisor node owns routing and the shared conversation state, and dispatches to worker subgraphs that each expose a narrow typed contract. Workers do not see each other's internal state, only what the supervisor passes, which keeps team ownership boundaries clean. Each worker has its own timeout, retry policy, and tool allowlist, so a badly-behaved worker degrades its own path rather than the whole run. The supervisor handles escalation to a human when confidence is low or a worker exhausts retries.

Q3. How did you make it safe for a junior engineer to contradict you in a room with senior people?

A. Mostly by structuring the session so contradiction was the assignment. I presented options rather than a preferred design, asked for holes rather than opinions, and called on the less senior people before the more senior ones, because the order of speaking in a room like that determines what gets said. When he made the point, I did not soften it into a hybrid of his idea and mine; I said his was better and we should build it, and I named him in the design doc.

Q4. Did open-sourcing part of it create risk?

A. Some, and it went through a security and legal review before it shipped. The concrete checks were that no internal endpoints, tenant configuration, or prompts with business logic were in the published surface, and that we were not exposing an interface we would be embarrassed to be held to. The side effect was that the interfaces got noticeably cleaner in the two weeks before publishing, because the team knew it would be read by people with no context.

CORE VALUE 6 OF 10

Take Risks, Remain Calm

Story: *Running brownout and fault-injection exercises, including a bounded production run, before signing a 99.99% commitment*

Situation. At Sprouts AI we were about to sign a 99.99% availability commitment for a tier-one agentic service. Every resilience test we had ever run was binary: dependency up, or dependency down. We had never tested what happened when a dependency was merely slow, which is the way most real cloud failures actually present. Grey failure was our blind spot and we were about to contractually commit around it.

Task. I proposed running brownout and controlled-degradation exercises, deliberately injecting latency and partial failures, and I proposed including a bounded exercise against production during a low-traffic window. Asking to intentionally degrade production before a big commitment is not a comfortable request to make.

Action.

- Started in staging so the risky part came last. Injected dependency latency, throttled traffic, and simulated partial failures, writing down expected behavior versus observed behavior for every scenario before running it.
- Found a serious defect immediately: our LLM provider client had a 60-second default timeout while our request budget was five seconds. Under provider slowness, worker threads pooled up on doomed requests and the entire service degraded, including paths that had nothing to do with the LLM.
- Fixed the failure modes rather than just documenting them: per-dependency timeouts that summed within the request budget, retries with exponential backoff and jitter, circuit breakers on each downstream, and bulkheads isolating the agent path from the read path.
- Added request shedding for low-priority traffic under saturation and idempotency plus fallback workflows so a shed or failed request could be safely retried by the caller.
- Designed the production exercise to be boring: five percent of traffic, a low-traffic window, a written abort criterion, a single kill switch, comms to support and the customer-facing team in advance, and me on the call watching dashboards with a second engineer whose only job was to call the abort.
- Put error-budget burn rate, p95 and p99 latency, saturation, and recovery time on dashboards with alerts, so the results became continuous monitoring rather than a one-time report.

Result. We found and fixed four latent failure modes before any customer met them, and sustained 99.99% for the service. Two months later a real provider degradation hit; the circuit breaker shed load and customers experienced a slower, degraded but functioning product instead of an outage, and the incident closed without a customer escalation. What I would generalize: calm under a risky change is not a temperament, it is preparation. The abort criteria and the kill switch are what made the risk reasonable to take.

Likely follow-up questions

Q1. What exactly was your abort criterion?

A. Written in advance and specific: abort if p99 latency on the non-injected control path exceeded twice baseline for more than two minutes, if error rate on any path crossed one percent, or if any customer-reported issue arrived through support during the window. The second engineer owned the call, not me, precisely because I was the one invested in the exercise succeeding and would have been slower to stop it.

Q2. How did you implement bulkheads concretely?

A. Bounded concurrency per dependency using semaphores, so the agent path could not consume every worker slot, plus separate connection pools rather than one shared pool. For the heaviest isolation we ran the batch and serving workloads as separate deployments on separate node pools, so resource contention was handled by the scheduler rather than by hope. The principle is that saturation of one dependency should be locally observable and locally contained.

Q3. What did you shed, and how did you decide priority?

A. We tiered requests: interactive user-facing agent calls were tier one and never shed, background enrichment and re-embedding jobs were tier two, and speculative pre-computation was tier three and shed first. The rule I used was whether a human was waiting on the result. Shedding is only acceptable if the caller can retry safely, which is why the idempotency work was a prerequisite rather than a nice-to-have.

Q4. What if the production exercise had caused a customer-visible outage anyway?

A. Then it would have been mine, and I would have said so in the postmortem in exactly those words. I had pre-written the customer communication before the window, because writing it under pressure is how you get it wrong, and recovery was a single switch that had been tested in staging. I also would not have hidden the fact that we chose to run it. My honest position is that a failure found by us in a five-percent window is enormously cheaper than the same failure found by a customer at peak.

CORE VALUE 7 OF 10

Own Without Ego

Story: *Being wrong about the OKE deployment topology at Sprouts AI, saying so in writing, and rebuilding on the reviewer's design*

Situation. I designed the Oracle Kubernetes Engine topology for our GoLang and Python services at Sprouts AI: a single node pool with shared workloads and one instance principal for all services. I had optimized for cost and operational simplicity, and I was fairly attached to it. In design review, a staff engineer said two things I did not want to hear: batch embedding jobs would starve latency-sensitive agent traffic on a shared pool, and a single instance principal across services violated least privilege. My first reaction in the thread was to defend the design.

Task. Get to the correct answer rather than to a defensible one. The uncomfortable part was that I had already socialized my design with the team and had some ownership invested in it being right.

Action.

- Stopped arguing in the thread and proposed a test instead, because the disagreement was empirical and we were treating it as a matter of opinion.
- Ran a load test with an embedding backfill executing concurrently against the agent path. p99 on the agent path went from roughly 600 milliseconds to about four seconds. His concern was not theoretical, it was the graph.
- Said plainly in the same review thread that I was wrong and that we were changing the design, in writing where the team could see it, rather than quietly revising the doc.
- Rebuilt the topology: separate node pools for batch and serving with taints and tolerations, resource requests and limits on every workload, pod disruption budgets, and independent autoscaling per pool since the two have completely different scaling profiles.
- Replaced the shared identity with per-workload instance principals, scoped through dynamic groups matching on compartment and tag, with policies written per compartment and nothing granted at the tenancy level. Each service got only the Object Storage buckets and Vault secrets it actually needed.
- Asked him to co-review the revision and credited the topology change to him in the design document.

Result. p99 stayed under a second on the agent path with batch jobs running concurrently, which was the whole point. The least-privilege pattern became the template for new services on the team, and a subsequent IAM review found no over-broad policies in that compartment. The cheapest thing I have ever done for a project's timeline is admit a design was wrong in week one instead of defending it to week six.

Likely follow-up questions

Q1. Walk me through how you scoped the instance principals.

A. Dynamic groups with matching rules on compartment OCID plus a workload tag, so membership was derived from where the instance ran and what it was, not from a hand-maintained list. Policies were written at the compartment level and each granted a specific verb on a specific resource type, for example manage

objects in one named bucket rather than manage object-family in the tenancy. Nothing was granted at the root compartment. The test I applied to each policy was whether I could explain why a narrower one would break something.

Q2. Why not just run one larger node pool and rely on requests and limits?

A. Requests and limits help with CPU and memory contention but they do not fix the mismatch in scaling profile. Serving needs headroom and fast scale-out on latency signals; batch is throughput-oriented, tolerant of queueing, and a good fit for cheaper preemptible capacity. One pool forces you to size for the union of both, so you either over-provision constantly or accept contention at the worst moment. Separating them was also cheaper in the end, which was the argument my original design had been built on.

Q3. Have you ever been on the other side of that conversation, as the reviewer?

A. Yes, and this experience changed how I do it. I now try to state the failure mode as a testable prediction rather than a judgment, because 'this will starve under concurrent batch load' invites a test while 'this design is wrong' invites a defense. I also try to give the author the chance to run the test themselves, since finding it yourself is a very different experience from being told.

Q4. Was there a cost to admitting it publicly?

A. Less than I expected, and I think I had the cost backwards. What actually damages credibility is defending something after the data has arrived. Being visibly correctable made subsequent design reviews faster, because people stopped hedging with me and started leading with their real objection.

CORE VALUE 8 OF 10

Earn Trust, Give Trust

Story: *Breaking the two-person incident hero pattern at Resilience Inc by handing real authority to newer engineers*

Situation. When I joined the platform team at Resilience Inc, incidents were handled by two engineers who held everything in their heads. Everyone else was effectively locked out, mean time to resolution was high, and the relationship with product had degraded to the point that stakeholders assumed engineering was concealing the real state of things. On the engineering side, nobody wanted to go on call because getting it wrong felt career-relevant.

Task. Reduce MTTR, but the underlying problem was trust in both directions: stakeholders did not trust our reporting, and engineers did not trust that a mistake during an incident would be survivable.

Action.

- Earning trust outward first: instrumented request rate, error rate, and duration per service, added distributed tracing across the Kafka path, and linked every alert directly to the dashboard that would confirm or deny it, so responding did not depend on knowing where to look.
- Made incident communication public and predictable. A status channel updated every twenty minutes during an incident, including updates that said we had no new information, because silence is what stakeholders interpret as concealment.
- Published blameless postmortems with real timelines, including our own bad decisions during the response. The first one I wrote was about a mistake of mine, deliberately, to establish the register.
- Giving trust inward: expanded the rotation to include newer engineers, but paired each with a shadow for their first cycles and wrote runbooks that contained actual commands rather than descriptions of commands.
- Gave new responders real authority: they could page anyone including me, and they could roll back production without approval. Authority you have to ask permission to use is not authority.
- Held back from overriding their calls in the moment even when I would have done something different, and debriefed afterward instead. That was the hardest part and the part that mattered most.

Result. MTTR dropped 40%, and the rotation grew from two people to seven. Product stakeholders stopped escalating around the team, because the status channel was faster than escalating. Two of the engineers who started as shadows became the primary responders I would want on a bad night. The thing I would tell anyone is that trust is given before it is earned, and people generally rise to the authority you actually hand them rather than the authority you describe.

Likely follow-up questions

Q1. What happened when one of them made the wrong call during a real incident?

A. It happened. A newer responder rolled back a release that turned out not to be the cause, which cost us about twenty minutes. In the channel and in the postmortem I backed the decision as reasonable given what was visible at the time, because it was. Privately afterward we walked through the two signals that would have

distinguished the cases. If I had corrected him publicly, the next person would have hesitated instead of acting, and hesitation costs far more than twenty minutes.

Q2. What makes a runbook actually usable at three in the morning?

A. Structure it by symptom, not by system, because at three in the morning you have a symptom. Each entry goes: what you are seeing, the dashboard link that confirms it, the exact command to verify, the exact command to remediate, what to check afterward to confirm it worked, and who to escalate to with their actual contact path. Copy-pasteable commands, no placeholders left for the reader to figure out. We also required that any runbook step that had not been executed in ninety days got exercised in a game day, because untested runbooks rot silently.

Q3. MTTR is one number. How did you know trust itself had improved?

A. I tracked adjacent signals. The count of stakeholder escalations that bypassed the on-call channel went to near zero. Rotation participation went from two people to seven voluntarily. And in retrospectives I asked directly whether people felt they could roll back production without checking with someone first, which is a more honest proxy for perceived authority than asking whether they feel trusted.

Q4. How do you keep a postmortem blameless when someone genuinely made an error?

A. Write the action, not the person, and then ask what made the action look correct at the time, because it almost always did. If an engineer deployed without checking a dependency, the finding is that the deploy path did not surface dependency state, not that the engineer was careless. That is not softening the truth; it produces a fix that survives that person leaving. The one place I do not stay neutral is on repeat patterns, which I raise directly with the individual outside the postmortem.

CORE VALUE 9 OF 10

Take Pride in Your Work

Story: *Rebuilding a nightly manual reconciliation at Tata Steel that nobody had asked me to touch*

Situation. At Tata Steel I owned integration-heavy enterprise platforms including Learning and Development, IRIS, and Coaching. The L and D platform had a nightly reconciliation that an operations team member performed by hand: exporting spreadsheets, re-keying data into the HR system, and eyeballing mismatches, roughly three hours every night. It was stable, nobody had complained, and it was not on any roadmap.

Task. Nobody asked me to change it. But I could not look at a system I owned that required a person to do robot work every night and feel that it was finished. I decided to make the case and then do it properly.

Action.

- Spent a week shadowing the operations process end to end and mapping every manual touchpoint, then quantified it: annual hours, observed re-keying error rate, and the audit exposure of an unlogged manual data path. That turned a matter of taste into a business case.
- Rebuilt the integration path as asynchronous Spring Boot services with idempotent batch jobs, so a failed run could simply be re-run rather than requiring someone to determine how far it had got.
- Replaced spreadsheet-shaped data transfer objects with an actual domain model, and applied SOLID principles and dependency injection so each integration had one clear seam that could be tested in isolation.
- Added contract tests against the downstream HR system so schema drift on their side broke our build instead of breaking a person's night.
- Changed the human's role rather than removing it: reconciliation reports now surface only true exceptions requiring judgment, instead of asking someone to find them in a full export.
- Raised test coverage on the critical path, wrote developer documentation, and had the operations team sign off on the exception workflow before cutover rather than after.

Result. Manual processing time dropped roughly 60% across the platforms, and the nightly exception list became small enough to clear in minutes. API response times improved about 40% at 99.99% uptime as a side effect of the rework. The codebase became the one we pointed new engineers at as the reference implementation, which I am prouder of than the metric. My standard since then is that 'it works' is not the finish line, it is the point at which the work becomes evaluable.

Likely follow-up questions

Q1. How did you justify the effort when nobody had asked for it?

A. By translating it into terms the business already tracked: operator hours per year, the error rate I observed during shadowing and what a mis-keyed training compliance record costs in a regulated environment, and the fact that a manual data path had no audit trail. I also scoped it so the first increment was three weeks rather than a quarter, which made the yes cheap. Nobody funds a crusade; people will fund a bounded experiment with a number attached.

Q2. What specifically made the result maintainable rather than just working?

A. Seams and tests, mostly. Each integration had one interface where the external system entered, so a change in that system touched one file. Contract tests meant downstream drift failed loudly in CI. Naming matched the vocabulary the operations team used, so a new engineer could map code to conversation. And no class did more than one thing, which sounds like a platitude until you are on call and need to reason about a single failure path.

Q3. Did you over-engineer anything?

A. Yes. I built a plugin abstraction for integration handlers anticipating a fourth and fifth system that never arrived. It had exactly one implementation for six months. I removed it during a later refactor and the code got simpler. Taking pride in work includes deleting things you were proud of writing, and I now try to wait for the second real case before generalizing.

Q4. How did you handle the operations team, given automation could look like a threat to their jobs?

A. I involved them from the shadowing week onward and was straightforward that the goal was to remove the re-keying, not the role. Their judgment on genuine exceptions was the part that actually needed a human, and the new workflow made that the job. They designed the exception report format with me and signed off before cutover. They ended up being the change's strongest advocates, which would not have happened if I had built it and then presented it.

CORE VALUE 10 OF 10

Challenge Ideas, Champion Execution

Story: *Challenging 'internal-only is safe' on a multi-tenant platform at Tata Steel, then owning the remediation to closure*

Situation. A multi-tenant enterprise platform at Tata Steel was being scoped to onboard a new business unit. The prevailing position, held by people more senior than me, was that because the platform was internal-only, tenant isolation could be handled in the application layer with a customer identifier in the query filter, and encryption at rest was unnecessary because the database sat behind the firewall. It was a reasonable-sounding position and it was the path of least work.

Task. I did not believe internal-only constitutes a security boundary. But an objection based on principle would have lost, and deserved to. I needed rigor rather than an opinion, and then I needed to own the alternative rather than just block the plan.

Action.

- Asked why first, in a real way: I wanted to understand the reasoning before attacking it, and part of it was sound, specifically that the threat model excluded external attackers.
- Threat-modeled the data flows and wrote a two-page document listing concrete scenarios rather than abstractions: a single missing WHERE clause exposing another business unit's payroll-adjacent data, database backups landing on a shared file server, and contractors with production read access.
- Tested the claim instead of asserting it. Wrote a sweep that exercised endpoints for tenant-filter omissions and found real instances in the existing codebase. That single result ended the debate, because it was no longer hypothetical.
- Presented findings without blame, framed as a class of defect the architecture made easy to introduce rather than as anyone's mistake.
- Proposed a graded plan rather than a rewrite: enforce isolation below the application through row-level policy plus a repository layer that structurally could not issue an unscoped query, add role-based access controls, encrypt in transit and at rest including backups, move secrets out of configuration files, and add audit logging.
- Then championed the execution rather than leaving a document behind. Sequenced the remediation by exploitability and blast radius, took the repository-layer rework myself because it was the hardest and least popular piece, and tracked every item to closure with a named owner and a date.

Result. Every identified gap was closed before the new business unit onboarded, and the repository pattern made the entire class of unscoped-query bug structurally impossible rather than a matter of reviewer vigilance. The platform passed internal audit without findings in that area. The lesson that stuck: challenging an idea only counts if you also own delivering the alternative, otherwise you have just added risk to someone else's plan and gone home.

Likely follow-up questions

Q1. How do you challenge a more senior person's position without it becoming a personal conflict?

A. Ask why before you disagree, and mean it, because roughly half the time you find a constraint you did not know about. Then bring evidence rather than judgment, and attack the idea rather than the person's competence. The thing that made this land was that I arrived with a plan, not just an objection. Someone who blocks without offering a path is expensive to have in a room; someone who blocks and then does the hard part is worth listening to next time.

Q2. Why enforce tenant isolation below the application layer instead of in queries?

A. Because a filter in a query is a discipline requirement and disciplines fail at scale. It only takes one new engineer, one hotfix at midnight, or one clever join to lose it, and the failure is silent. Pushing it into a repository layer that cannot construct an unscoped query, backed by row-level policy in the database, makes the correct behavior the default and the incorrect one hard to express. Defense in depth also means the application bug and the database policy have to fail together for data to leak.

Q3. How did you sequence the remediation?

A. Ordered by exploitability multiplied by blast radius, with a bias toward two early quick wins for momentum. Secrets out of configuration and encryption at rest went first because they were low-effort and high-visibility. The repository rework came next as the largest item, sequenced ahead of onboarding since retrofitting isolation after a second tenant is in the data is far harder. Audit logging came last because it detects rather than prevents. Every item had one named owner and a date, and I reviewed the list weekly until it was empty.

Q4. What would you have done if leadership had said no?

A. Two things. I would have made sure the decision was documented with the specific risks accepted and by whom, because an accepted risk is legitimate and an undocumented one is not. Then I would have asked for the narrowest subset I could get, probably the repository layer, since it was cheap and prevented the highest-severity scenario. Disagree and commit is real, but committing does not mean pretending the risk register is empty.

Closing Notes

Delivery checklist

- Open with one sentence of context, then the tension. Interviewers stop listening if the setup runs past 20 seconds.
- Say 'I' when describing your actions and 'we' when describing the team's outcome. Loops at this level are explicitly listening for which one you use.
- Every Result ends with a number. If you cannot remember the number, say the direction and the magnitude honestly rather than inventing a figure you will be asked to defend.
- Close each story with the lesson. It is the single easiest way to signal seniority and it maps directly to how OCI frames learning from failure.
- Respect confidentiality. Describe architecture, mechanisms, and your own metrics; do not name customers, share internal documents, or quote unpublished numbers you agreed to protect.
- When a question is ambiguous, ask which aspect they want before launching. OCI's own technical guidance rewards clarifying questions.

Questions worth asking your interviewers

- How are SLOs and error budgets set for the components this role owns, and what actually happens operationally when a budget is exhausted?
- What does the on-call rotation look like for this team, and how is operational load balanced against feature delivery?
- Where is the boundary between this team's components and adjacent teams', and how are cross-team design disagreements resolved?
- What does the change-management path look like for a risky infrastructure change here, and how much of it is automated today?
- For someone joining at Principal level, what would you want them to have delivered or improved by the end of the first six months?
- Which of the OCI core values does this team find hardest to live up to in practice, and why?

Traps to avoid

- Reusing one story for two values. The index on page one exists to prevent this; commit it to memory before the loop.
- Describing a team accomplishment without making your own contribution unmistakable.
- Answering 'tell me about a failure' with a disguised strength. Use the OKE topology story; it is a real error with a real cost and a clean recovery.

- Drifting into tools and frameworks when asked about impact. Name the mechanism, then return immediately to the customer or reliability outcome.
- Over-claiming scope. If you influenced rather than owned something, say so; it reads as more senior, not less.